Kaggle Competition Market Prediction Blueprint

computer science (Vivekanand Education Society's Institute of Technology)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by ?? ? (gaolijin0807@gmail.com)

The Kaggle Quant Competition Playbook: A Grandmaster's Blueprint

Problem Understanding: Deconstructing the Competition Mandate

A precise and nuanced understanding of the competition's objective, evaluation metric, and constraints is the bedrock upon which a winning strategy is built. Misinterpreting any of these foundational elements is the most common and fastest path to failure in quantitative finance competitions. The task must be approached not merely as a standard machine learning problem, but as a constrained optimization challenge set within a financial context.

The Core Objective: Maximizing a Custom Volatility-Adjusted Sharpe Ratio

The central goal of this competition is to develop a model that generates a daily investment position, designated as prediction, which can range from 0 (fully divested) to 2 (maximum allowed investment) in the S&P 500. The success of this model is not measured by its raw predictive accuracy but by its ability to maximize a custom Sharpe ratio.

The Sharpe ratio is a widely used financial metric for calculating risk-adjusted returns.[1] It quantifies the performance of an investment by comparing its returns to those of a risk-free asset, after accounting for the investment's risk or volatility.[2] The standard ex-ante Sharpe ratio is defined by the formula:

$$S=\sigma pE$$

where E is the expected value of the excess return of the portfolio over the risk-free rate, and σp is the standard deviation of the portfolio's excess returns.[2] A higher Sharpe ratio indicates a better return for the same amount of risk.

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

Crucially, this competition is not a pure forecasting exercise. It is a problem of *tactical allocation*. The model's output is not a predicted return value but a decision—an investment level—that directly constitutes a portfolio strategy. The evaluation metric then scores the historical performance of this generated portfolio.

The Machine Learning Task: A Time-Series Regression for Tactical Allocation

At its core, the problem is a time-series regression task. The model is provided with a comprehensive set of features up to a given day, t, and is tasked with predicting the forward_returns for the subsequent day, t+1. The dataset is structured as a rich, multi-factor time series, with features categorized into distinct families: Market (M*), Economic (E*), Interest Rate (I*), Price (P*), Volatility (V*), Sentiment (S*), Momentum (MOM*), and Dummy (D*) variables.[5] This structure is highly characteristic of datasets used in institutional quantitative finance and mirrors those seen in past competitions like the Jane Street Market Prediction challenge.[6]

The final submission, the prediction value, will be a direct function of the model's forecasted forward_returns. A simple, naive mapping might be to invest fully (prediction = 2) if the predicted return is positive and divest completely (prediction = 0) otherwise. However, as the metric analysis will show, a more sophisticated mapping is required for success.

Deep Dive into the Target Metric: Exploiting the Nuances of the Sharpe Ratio and Its Penalties

A meticulous analysis of the competition's custom evaluation metric, as revealed in provided code [8] and reinforced by analyses in public notebooks [9], uncovers the single most critical strategic element of the competition: its penalty structure. The metric is not a simple Sharpe ratio; it is an

adjusted Sharpe ratio with conditional penalties.

- **Volatility Penalty:** A penalty term, vol_penalty, is applied to the Sharpe ratio, but *only if* the strategy's annualized volatility surpasses the market's annualized volatility by a factor of 1.2. That is, a penalty is incurred if strategy_volatility / market_volatility > 1.2. If the strategy's volatility remains within this 20% corridor, no penalty is applied.
- **Return Penalty:** A separate penalty is applied *only if* the strategy's mean excess return

Downloaded by ?? ? (gaolijin0807@gmail.com)

falls below the market's mean excess return. In other words, underperforming the market on a return basis is penalized.

This structure leads to a profound strategic conclusion. If a competitor's strategy successfully stays below the 1.2x volatility threshold and simultaneously outperforms the market's average return, both penalty terms become inactive. In this scenario, the complex custom metric simplifies to the standard, unadjusted Sharpe ratio. This fundamentally transforms the problem from simply maximizing a score into a constrained optimization task: maximize the standard Sharpe ratio, subject to the hard constraint that strategy_volatility <= 1.2 * market_volatility.

This constraint-driven objective has immediate and far-reaching implications for model design. The competition is not about minimizing a common loss function like Root Mean Squared Error (RMSE). A model that achieves a low overall prediction error but fails to identify high-confidence opportunities will be inferior to a model that may have a higher average error but is exceptionally accurate in predicting the *sign* of large market moves. The latter allows for a strategy that is aggressively invested on strongly positive days and fully divested on negative days, thereby maximizing the numerator of the Sharpe ratio (mean return) while managing the denominator (standard deviation).

However, a strategy of being fully invested on *every* day with a predicted positive return, no matter how small, risks accumulating excessive volatility and breaching the 1.2x cap. This leads to a more refined understanding of the task: the model must produce not only a point forecast of the return but also an implicit or explicit *confidence score* for that forecast. This confidence can then be used to scale the investment level. For instance, predictions in the top decile of confidence might warrant a full investment of 2.0, while predictions in lower positive deciles might receive a scaled-down investment (e.g., 1.0 or 1.5). This practice, known as volatility targeting, is a cornerstone of quantitative risk management and is explicitly incentivized by the competition's metric structure.[9] The task is therefore not just to build a predictor, but to build an integrated prediction and risk management system.

Rules of Engagement: Navigating Constraints, Allowances, and the Time-Series API

This competition is structured as a code competition, utilizing a time-series API for submissions. This format, similar to the one used in the Jane Street Market Prediction competition [7], has several critical implications:

- **No Forward Peeking:** The API strictly enforces the temporal nature of the problem. When the model is making a prediction for a given day, it can only access data from that

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

day and prior days. This makes a robust, time-aware local validation framework absolutely essential to prevent developing models that inadvertently learn from future information.

- **Runtime Constraints:** The execution of the submission notebook will be subject to strict time limits, such as a 5-hour cap per phase.[7] This constraint directly influences choices regarding model complexity, the computational cost of feature engineering, and the feasibility of large ensembles.
- **External Data Allowance:** Competitions of this nature typically permit the use of external data, provided it is freely and publicly available.[7] This represents a significant opportunity to introduce unique, predictive signals that are not present in the base dataset, creating a substantial competitive advantage.

Data Analysis Roadmap: From Raw Data to Actionable Intelligence

In a domain as notoriously signal-poor as financial market prediction, sophisticated feature engineering is not merely a preparatory step; it is the primary arena where the competition is won or lost. This section outlines a systematic roadmap for transforming the raw dataset into a rich set of actionable, predictive features.

The Data Landscape: A Multi-Factor Financial Time-Series Dataset

The provided data is tabular and organized as a time series, indexed by a date_id and containing a large number of anonymized features.[5] Key characteristics include:

- **Factor Families:** The features are logically grouped into families denoted by prefixes (M, E, I, P, V, S, MOM, D). Despite the anonymization of individual features, these groupings provide invaluable domain context, hinting at their underlying economic nature (e.g., Market, Economic, Volatility).[5]
- **Missing Data:** The dataset exhibits a significant degree of missingness, with approximately 15.6% of cells being empty.[5] This necessitates a carefully considered imputation strategy that respects the time-series nature of the data.
- **Target Variable:** The target, forward_returns, is distributed around a mean close to zero, with a slight negative skew and kurtosis indicating lighter tails than a normal distribution. This is a typical statistical profile for daily asset returns.[5]

Downloaded by ?? ? (gaolijin0807@gmail.com)

A Systematic EDA Protocol: Uncovering Temporal Patterns, Stationarity, and Feature Relationships

A rigorous Exploratory Data Analysis (EDA) protocol is the first step in understanding the data's structure and potential. The following steps should be undertaken:

1. **Missing Value Analysis:** Investigate the patterns of missing data over time. A plot of missing value counts per date_id can reveal if certain periods (e.g., market crises, holidays) have more missing data. Analyzing missingness by feature family can also inform imputation choices. For instance, persistent, slow-moving features might be best handled with a forward-fill, while more volatile features might require interpolation or model-based imputation.[5]
2. **Target Distribution and Dynamics:** Examine the time series of forward_returns. Specifically, look for evidence of **volatility clustering**—a well-documented stylized fact of financial returns where periods of high volatility are followed by more high volatility, and calm periods are followed by calm.[5] Visualizing the rolling standard deviation of returns will make this apparent. This observation suggests that features derived from volatility models like GARCH could be highly predictive.
3. **Feature Stationarity:** For each feature, perform statistical tests for stationarity, such as the Augmented Dickey-Fuller (ADF) test. Non-stationary time series, which have statistical properties like mean and variance that change over time (e.g., a stock price level that trends upwards), can lead to spurious correlations and unreliable models. Such features must be transformed into a stationary form, typically by taking first differences or calculating percentage changes (returns), before being used in most machine learning models.[11]
4. **Correlation and Redundancy Analysis:** Compute and visualize the correlation matrix of the features. Identify blocks of highly correlated features (e.g., different tenors of interest rates). This multicollinearity can make model interpretation difficult and suggests that dimensionality reduction techniques like Principal Component Analysis (PCA) on these blocks could create more robust, orthogonal features.[13]
5. **Time-Based Visualization:** Plot key features and their rolling statistics (e.g., 50-day moving average) over time. This can help identify long-term trends, cyclical patterns, or structural breaks in the data that may correspond to major economic events.

Table 1: Advanced Feature Engineering Playbook for Financial Time Series

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

The following table provides a structured playbook for feature creation. This systematic approach, drawing from established quantitative finance and machine learning practices, is crucial for building a powerful and reproducible feature set.

Category	Feature Description	Implementation Detail	Rationale & Key Sources
Lag & Rolling Statistics	Lagged values of key features (e.g., feature_X_lag_1).	df['X'].shift(1)	Provides the model with direct access to past information, essential for tree-based models that are not inherently sequential. [16]
	Rolling window statistics (mean, std, skew, kurtosis) over various horizons (e.g., 5, 20, 60 days).	df['X'].rolling(20).mean()	Smooths noisy data, captures local trends and changes in distribution, and provides context about recent market behavior. [16]
Momentum & Trend	Relative Strength Index (RSI).	ta.momentum.RSIIndicator(close=df['Close'], window=14).rsi()	Measures the speed and change of price movements to identify overbought or oversold conditions. A classic momentum indicator. [18]
	Moving Average Convergence Divergence (MACD).	ta.trend.MACD(close=df['Close']).macd_diff()	Captures the relationship between two moving averages of

Downloaded by ?? ? (gaolijin0807@gmail.com)

			a price series to signal changes in trend and momentum. [18]
	Moving Average Crosses (e.g., 50-day vs. 200-day).	np.where(df > df, 1, 0)	Simple yet powerful trend-following signals. The "golden cross" and "death cross" are widely followed market indicators. [19]
Volatility & Risk	Historical Volatility (Realized Volatility).	df['returns'].rolling(21).std() * np.sqrt(252)	Direct measure of recent risk. Volatility clustering implies that recent high volatility predicts future high volatility. [5]
	GARCH(1,1) Forecasted Volatility.	Train a GARCH model on historical returns and use it to predict the next day's conditional volatility.	GARCH models are the standard for modeling time-varying volatility in finance. This feature provides a sophisticated, forward-looking estimate of risk. [20]
Market Microstructure Proxies	Amihud Illiquidity Measure.	abs(df['returns']) / df['dollar_volume']	A proxy for price impact. High values suggest that trading volume has a large effect on price, indicating illiquidity. [23]
	Advance-Decline Ratio.	advancing_stocks / declining_stocks	Measures market breadth. A rising

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

		(requires external data)	ratio indicates broad market participation in an uptrend, which is a healthy sign. [19]
Data Transformation	Principal Component Analysis (PCA) on correlated feature groups.	PCA(n_components=3).fit_transform(df[['feat1', 'feat2',...]])	Reduces dimensionality and creates orthogonal factors from redundant features (e.g., transforming multiple interest rate features into 'level', 'slope', and 'curvature' factors). [14]
	Discrete Wavelet Transform (DWT).	pywt.dwt(df['feature_X'], 'db1')	Decomposes a time series into different frequency components, which can help to de-noise the signal and separate short-term fluctuations from long-term trends. [14]

The explicit categorization of features into families (M*, E*, etc.) is a significant clue provided by the competition organizers.[5] It suggests that the market does not behave monolithically; rather, it transitions through different states or

regimes, where different types of information become more or less important. For instance, during a stable economic expansion, market (M*) and momentum (MOM*) features might dominate. In contrast, during a recession or crisis, macroeconomic (E*) and volatility (V*) features may become paramount.

This points toward a powerful feature engineering strategy: explicitly modeling the market regime. This concept was a key element in winning solutions for the Two Sigma Financial Modeling Challenge, where top competitors partitioned data by volatility and trained distinct models for each regime.[25] A sophisticated way to implement this is to first create a "regime" variable. This can be done by applying an unsupervised clustering algorithm, such as a

Downloaded by ?? ? (gaolijin0807@gmail.com)

Gaussian Mixture Model (GMM), to a subset of key state-defining features, primarily from the volatility (

V*) and macroeconomic (E*) families.[26] The GMM might identify, for example, three distinct market states: a "Low-Volatility Bull" regime, a "High-Volatility Bear" regime, and a "Transitional" regime.

The output of this clustering—a categorical variable indicating the current regime for each day—can then be included as a powerful feature in the main predictive model. This allows a single, flexible model like LightGBM to learn different feature interactions and decision boundaries for each market state, effectively creating an implicit, dynamic regime model. This approach directly translates a proven winning strategy from past competitions into a concrete, actionable feature.

Baseline Approach: Securing a Foothold on the Leaderboard

Establishing a strong, simple, and reliable baseline model is a critical early-game objective. It serves to validate the entire data processing and evaluation pipeline, provides a robust benchmark against which more complex models can be measured, and facilitates an early presence on the competition leaderboard.

The Workhorse Model: Building a Robust LightGBM Baseline

LightGBM, a gradient boosting framework, is an ideal choice for a baseline model. It is renowned for its exceptional performance on tabular data, its computational speed, and its memory efficiency, making it a dominant tool in the Kaggle ecosystem.[16]

Since LightGBM is not an inherently sequential model, the time-series data must be transformed into a supervised learning format. This is achieved through the feature engineering process described previously, particularly the creation of lag and rolling window features.[16] These features explicitly provide the model with a view of the recent past, allowing it to learn temporal patterns. The initial baseline should be built using a curated subset of 20-30 of the most promising features identified during EDA to ensure rapid iteration and debugging.

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

Optimizing the Baseline: The Critical Role of a Rigorous Validation Framework and Feature Importance Analysis

A baseline model is only as good as the framework used to evaluate it. From the very first run, a rigorous validation strategy is non-negotiable.

- **Time-Aware Cross-Validation:** A proper time-series cross-validation scheme must be used. A simple starting point is the TimeSeriesSplit from the scikit-learn library, which ensures that the validation set for each fold always occurs after the training set.[28] This prevents the model from learning from future data. This initial setup will later be upgraded to a more sophisticated method like Combinatorial Purged CV.
- **Early Stopping:** To prevent overfitting and to determine the optimal number of boosting iterations, the model should be trained with an early stopping mechanism. This involves monitoring the model's performance on a validation set and halting the training process when the score on this set ceases to improve for a specified number of rounds.
- **Feature Importance Analysis:** LightGBM provides built-in methods for calculating feature importance, typically based on "gain" (the total improvement to the objective function brought by a feature) and "split" (the number of times a feature is used to make a split in a tree). Analyzing these importance scores after each model run creates a powerful and tight feedback loop for development. Features that are consistently ranked as important can be further investigated, and new features can be engineered based on interactions or transformations of these key drivers. This iterative process of building, evaluating importance, and engineering new features is a core engine of progress in tabular data competitions.

Advanced Modeling Strategy: Ascending to the Top Ranks

Moving beyond the baseline requires the development of more sophisticated models capable of capturing complex, non-linear, and long-range dependencies in the data. The strategic goal is not to find a single "best" model, but to cultivate a diverse set of strong models whose different strengths and weaknesses can be combined in a powerful ensemble.

Downloaded by ?? ? (gaolijin0807@gmail.com)

Beyond Gradient Boosting: Sequential Deep Learning Architectures

While tree-based models excel at learning complex interactions from tabular features, deep learning models designed for sequential data can capture temporal dynamics that may be missed by gradient boosting.

- **LSTMs (Long Short-Term Memory):** LSTMs are a specialized type of Recurrent Neural Network (RNN) architected specifically to address the challenge of learning long-term dependencies in sequential data.[29] Unlike standard RNNs, which can struggle with vanishing or exploding gradients over long sequences, LSTMs employ a gating mechanism. This mechanism consists of three "gates"—the forget gate, input gate, and output gate—within each LSTM cell. These gates control the flow of information, allowing the network to selectively remember relevant information from the distant past and forget irrelevant data, making them well-suited for the persistent trends and regimes found in financial time series.[30] Architectures can range from a simple Vanilla LSTM with a single hidden layer to more complex Stacked LSTMs (multiple layers) or Bidirectional LSTMs that process the sequence both forwards and backwards to capture context from both past and future (relative to a point in the sequence).[32]
- **Transformers:** Originally developed for Natural Language Processing (NLP), the Transformer architecture has shown immense promise for time-series modeling.[33] Its core innovation is the **self-attention mechanism**, which allows the model to weigh the importance of all other data points in the input sequence when encoding a specific point. This is in stark contrast to RNNs, which process data sequentially. By processing the entire sequence at once and learning these attention weights, Transformers can be exceptionally effective at capturing very complex and long-range dependencies without the sequential processing bottleneck of RNNs.[35]
- **MLPs with Autoencoders:** A highly effective strategy, which secured first place in the Jane Street Market Prediction competition, involves using a supervised autoencoder as a non-linear feature extractor.[36] An autoencoder is a neural network trained to reconstruct its input, forcing it to learn a compressed, salient representation of the data in its central "bottleneck" layer. By training this autoencoder jointly with a downstream Multi-Layer Perceptron (MLP) that performs the final prediction, the model learns a data representation specifically optimized for the predictive task. The key to implementing this correctly in a time-series context is to train the autoencoder *within* each cross-validation fold to prevent any information from the validation set from leaking into the feature generation process.

Table 2: Comparative Analysis of Advanced Model Architectures

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

The selection of models should be a strategic decision based on their inherent strengths and how they complement each other within an ensemble.

Model	Strengths	Weaknesses	Strategic Use Case in this Competition	Key Sources
LightGBM	Fast, efficient, excellent with tabular data, handles missing values, provides feature importance.	Not inherently sequential; requires extensive feature engineering to capture time dynamics.	Primary model for rapid iteration, feature selection, and as a powerful component of the final ensemble.	[16]
LSTM	Explicitly designed for sequential data; excels at capturing long-term temporal dependencies through its memory cells.	Slower to train than tree models; can be sensitive to hyperparamet er choices.	Captures deep temporal patterns and auto-correlatio ns in the time series that may be missed by static, feature-based models.	[29]
Transformer	Superior at modeling very long-range and complex dependencies via self-attention; parallelizable training.	Can be data-hungry and computationall y expensive; may overfit on smaller datasets without careful regularization.	Models complex, non-local interactions across long time horizons, potentially identifying relationships that other	[34]

Downloaded by ?? ? (gaolijin0807@gmail.com)

			architectures cannot.	
AE + MLP	Learns a task-specific, non-linear feature representation automatically; proven winning architecture.	Adds model complexity; requires careful implementatio n to avoid data leakage during training.	Acts as a powerful automated feature engineering layer, creating a rich set of inputs for a simple but effective predictor.	36

The Art of the Ensemble: Blending and Stacking for Robustness and Performance

Virtually all winning solutions in modern Kaggle competitions are ensembles of multiple models. The underlying principle is that by combining the predictions of diverse models that make different types of errors, the combined prediction will be more accurate and robust than any single model.

- **Blending:** This is the simplest and often most effective ensembling technique. It involves taking a weighted average of the predictions from the best-performing models (e.g., a LightGBM, an LSTM, and a Transformer). The optimal weights for this average can be determined by optimizing the final ensemble's score on the out-of-fold validation predictions.
- **Stacking (Stacked Generalization):** This is a more sophisticated approach. It involves a two-level structure. In "level 1," several diverse base models are trained, and their out-of-fold predictions on the training data are saved. These predictions are then used as input features for a "level 2" model, often called a meta-model (e.g., a simple linear regression or another LightGBM). This meta-model learns the optimal way to combine the base model predictions, potentially capturing complex non-linear relationships between them.

Automated Excellence: Sophisticated Hyperparameter Optimization

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

with Optuna

Manually tuning the hyperparameters of complex models is inefficient and unlikely to find the optimal configuration. Modern Bayesian optimization frameworks like Optuna provide a more principled and automated approach.[37]

The process involves defining an objective function that encapsulates the model training and evaluation process. This function takes a trial object from Optuna, which is used to sample hyperparameters from predefined search spaces (e.g., trial.suggest_float('learning_rate', 1e-4, 1e-1, log=True)). The function then trains the model with these parameters and returns the score on the validation set—in this case, the custom volatility-adjusted Sharpe ratio. Optuna's study object then repeatedly calls this function, using efficient algorithms like Tree-structured Parzen Estimator (TPE) to intelligently explore the search space and converge on the set of hyperparameters that maximizes the objective score.

External Resources: Gaining a Competitive Edge

The provided dataset is the common ground for all competitors. Introducing unique, predictive information through external data is one of the most effective ways to create a competitive advantage.

Public Datasets for Feature Augmentation

Freely available public datasets can provide invaluable macroeconomic and market context that may not be fully captured by the anonymized features in the base dataset.

- **FRED (Federal Reserve Economic Data):** Maintained by the Federal Reserve Bank of St. Louis, FRED is an extensive database containing over 800,000 economic time series from dozens of sources.[40] For the task of predicting S&P 500 returns, relevant series include measures of inflation (Consumer Price Index - CPI), employment data (unemployment rate), industrial production, consumer sentiment indices, and various interest rate spreads (e.g., the spread between the 10-year and 2-year Treasury yields, which is a widely watched recession indicator).[41] These variables are crucial for building the market regime models discussed earlier.
- **VIX Index (Cboe Volatility Index):** Often referred to as the "fear index," the VIX

Downloaded by ?? ? (gaolijin0807@gmail.com)

measures the market's expectation of 30-day forward-looking volatility, derived from the prices of S&P 500 index options.[43] The VIX has a strong, historically negative correlation with S&P 500 returns; high and rising VIX levels are associated with market stress and downturns. The VIX level itself, its daily change, and rolling statistics of its level and change should be incorporated as high-priority features.

Leveraging Pre-trained Models and Insights from Academic Research

While pre-trained models are less common in quantitative finance compared to computer vision or NLP, the *architectures* and *methodologies* detailed in academic papers and the write-ups of past competition winners are invaluable resources.[35] Implementing a model architecture that has been proven effective on a similar task provides a significant head start. For example, the review of Transformers for time series highlights several efficient variants like Informer and Autoformer that could be adapted.[35]

Learning from the Shoulders of Giants: Essential Kaggle Kernels and GitHub Repositories

The Kaggle community is a rich source of shared knowledge. A thorough review of winning solutions from past financial prediction competitions is essential homework.

- Jane Street Market Prediction [7]:
  Key takeaways include the power of non-linear feature extraction via supervised autoencoders and the utility of multi-label classification.
- Two Sigma Financial Modeling Challenge [25]:
  This competition underscored the importance of modeling market regimes, the danger of overfitting the public leaderboard, and the need for robust, error-free code in API competitions.
- Optiver Realized Volatility Prediction [45]:
  This competition demonstrated that for high-frequency data, deep learning models (like 1D CNNs) applied to raw data sequences can outperform models based on handcrafted features.
- **Open-Source Libraries:** Repositories like mlfinlab on GitHub provide Python implementations of many advanced techniques from the influential book "Advances in Financial Machine Learning," including the crucial cross-validation methods discussed in the next section.

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

Evaluation & Validation: The Bedrock of a Winning Strategy

In a time-series competition with a hidden future test set, the single most important component of the entire project is the validation strategy. A flawed or overly simplistic validation scheme will produce misleading scores, foster a false sense of progress, and almost certainly lead to a dramatic failure on the private leaderboard. The goal is to create a local validation framework that mimics the final evaluation process as closely as possible.

The Gold Standard for Financial Time Series: Combinatorial Purged Cross-Validation

Standard cross-validation techniques are inappropriate and dangerous for financial time series.

- **Invalidity of K-Fold CV:** Standard K-Fold CV, which randomly shuffles and partitions the data, is completely invalid. It violates the temporal ordering of the data, leading to a situation where the model is trained on future data to predict the past—a form of data leakage that results in wildly optimistic and useless validation scores.[28]
- **Limitations of Walk-Forward CV:** A simple walk-forward or expanding window approach (like scikit-learn's TimeSeriesSplit) is a major improvement as it respects temporal order.[28] However, it can be statistically unstable, especially with a limited amount of data, as the size of the training set changes with each fold.

The state-of-the-art method for backtesting financial machine learning models is **Combinatorial Purged Cross-Validation (CPCV)**, as detailed by Marcos López de Prado.[48] This method is specifically designed to address the unique challenges of financial data. Its key components are:

1. **Combinatorial Splits:** The data is first divided into N sequential folds. Instead of a single walk-forward path, multiple train/test splits are created, allowing for a more robust and comprehensive evaluation.
2. **Purging:** This is the most critical step. Financial labels are often derived from information that spans a period of time. For example, if the target is the return over the next 10 days, the label for day t depends on the price at day t+10. If the test set starts at day t+1, a standard split would include training data from day t whose label is contaminated by

Downloaded by ?? ? (gaolijin0807@gmail.com)

information from within the test period. Purging involves identifying and removing all training data points whose labels overlap in time with the test set, thereby eliminating this source of data leakage.

3. **Embargoing:** Information in financial markets does not diffuse instantaneously. A major event at the end of the test set can have lingering effects (autocorrelation) on the data immediately following it. Embargoing involves placing a small gap by removing training samples that immediately follow the end of the test period, ensuring a cleaner separation between training and testing information.

Implementing a correct and robust CPCV framework is a top-priority, foundational task. All subsequent model development, feature selection, and hyperparameter tuning must be evaluated using this framework to generate reliable estimates of performance on the final, hidden test set.

Navigating Leaderboard Pitfalls: A Guide to Avoiding Overfitting, Data Leakage, and Metric Traps

- **Public Leaderboard Overfitting:** The public leaderboard score is calculated on a small, fixed portion of the test data. It is often not representative of the full test set and can be a siren's call, luring competitors into making decisions that improve their public rank at the expense of true generalization. This was a particularly painful lesson in the Two Sigma competition, where the public and private test sets covered different market regimes, causing massive leaderboard shake-ups.[25] The local CPCV score must always be trusted over the public leaderboard score.
- **Insidious Data Leakage:** Beyond the temporal leakage addressed by CPCV, other forms of leakage can creep into the pipeline. A common error is performing data preprocessing steps, such as scaling or normalization, on the entire dataset *before* splitting it into training and validation folds. All parameters for such transformations (e.g., the mean and standard deviation for standardization) must be learned *only* from the training data of a given fold and then applied to the validation data for that fold.
- **Metric Traps:** As established in the problem deconstruction, the evaluation metric is a constrained optimization problem, not a simple error minimization one. Optimizing for a proxy metric like RMSE or classification accuracy will lead to a suboptimal strategy. The entire model development and tuning process must be guided by the custom volatility-adjusted Sharpe ratio, calculated on the out-of-fold predictions from the CPCV framework.

Competition Strategy & Tips: Mastering the

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

Meta-Game

Exceptional technical skill is a necessary but not sufficient condition for winning a Kaggle competition. Top performance also requires a disciplined meta-strategy for managing time, iterating efficiently, collaborating effectively, and making shrewd submission decisions.

Operational Excellence: Best Practices for Code Reproducibility, Teamwork, and Fast Iteration

- **Code Reproducibility:** A scientific and reproducible workflow is paramount. Use a fixed random seed for all stochastic processes to ensure that results are repeatable. Employ version control (e.g., Git) for all code. Utilize experiment tracking tools (e.g., MLflow, Weights & Biases) to log the code version, parameters, and results of every single experiment. This creates an auditable and organized record of progress, preventing wasted effort and confusion.
- **Fast Iteration:** The speed of the development cycle is a key competitive advantage. The ability to test a new feature or model idea quickly allows for more experimentation. This can be achieved by creating a master "pipeline" script that automates the entire workflow: data loading, feature engineering, cross-validation, model training, prediction, and evaluation. For initial development and debugging, this pipeline should be run on a small, stratified sample of the data (e.g., 10-20%), allowing for iteration in minutes rather than hours.

Strategic Resource Allocation: When to Pursue Model Diversity vs. Deep Hyperparameter Tuning

Time is the most valuable resource in a competition. Allocating it effectively across different stages is crucial.

- **Early Game (Weeks 1-2):** The overwhelming focus should be on building a bulletproof validation pipeline (CPCV) and engaging in broad, creative feature engineering. The goal is to create a rich pool of potentially predictive signals.
- **Mid Game (Weeks 2-3):** The focus shifts to **model diversity**. The largest gains in performance typically come from ensembling models that are fundamentally different and thus make uncorrelated errors. Time should be spent building and validating distinct

Downloaded by ?? ? (gaolijin0807@gmail.com)

model architectures (e.g., a tree-based model, an RNN, a Transformer). A simple blend of two decent, diverse models will almost always outperform a single, perfectly tuned model.

- **Late Game (Week 4):** Only after establishing a portfolio of 2-3 strong, diverse models should the focus shift to intensive hyperparameter tuning using automated tools like Optuna. The final days should be spent optimizing the ensemble weights and running final, long training sessions on the full dataset.

The Final Push: Mastering Late-Stage Submissions through Blending and Bagging

The final submissions are the culmination of the entire effort and should be chosen strategically.

- **Submission Selection:** A common and effective strategy is to select two final submissions: one being the best-performing ensemble, and the other being the single best and most robust individual model (based on CV score and stability across folds). This approach hedges against the possibility that the private test set has characteristics that happen to favor the single model over the ensemble.[25]
- **Bagging (Bootstrap Aggregating):** A simple yet powerful technique to reduce model variance and eke out a final performance boost. This involves training the same final model architecture multiple times (e.g., 5-10 times) using different random seeds for initialization and data shuffling. The final prediction is then the average of the predictions from these bagged models. This almost always results in a small but meaningful improvement in score.

Step-by-Step 30-Day Battle Plan

A structured, disciplined plan ensures consistent progress, prevents last-minute panic, and maximizes the chances of success. This timeline allocates effort strategically across the four weeks of a typical competition.

Week 1: Foundation, EDA, and a Robust Baseline

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

The first week is dedicated to building the essential infrastructure and gaining a deep understanding of the problem.

- **Days 1-2:** Set up the coding environment, libraries, and version control. Perform a comprehensive EDA, focusing on the areas outlined in the Data Analysis Roadmap. Deeply analyze the custom evaluation metric and its implications.
- **Days 3-5:** Implement the Combinatorial Purged Cross-Validation (CPCV) framework. This is the highest priority task of the week. Thoroughly test and debug it to ensure it is functioning correctly and preventing data leakage.
- **Days 6-7:** Build, train, and validate a baseline LightGBM model using a small set of ~20-30 initial features. The goal is to have a complete, end-to-end pipeline that produces a reliable local CV score. This score will be the benchmark for all future work.

Week 2: Intensive Feature Engineering and Advanced Model Prototyping

With a solid foundation in place, the second week is focused on expanding the feature set and beginning to build model diversity.

- **Days 8-11:** Systematically work through the Feature Engineering Playbook (Table 1). Generate hundreds of features from all categories: lags, rolling statistics, momentum indicators, volatility measures, etc. Use the baseline LGBM's feature importance to guide this process and select the most promising candidates.
- **Days 12-14:** Begin development of a second, architecturally diverse model. An LSTM is a good candidate. The goal is not to perfect it, but to get a working version integrated into the CPCV framework and producing a reasonable score.

Week 3: Ensembling, Hyperparameter Tuning, and Strategic Experimentation

The third week is about combining models and starting the optimization process.

- **Days 15-17:** Create a simple weighted blend of the LightGBM and LSTM models. Optimize the blending weights on the out-of-fold validation predictions. The resulting ensemble score should demonstrate an improvement over either individual model.
- **Days 18-21:** Set up and launch automated hyperparameter tuning jobs for the primary

Downloaded by ?? ? (gaolijin0807@gmail.com)

models using Optuna. While these jobs run, begin prototyping a third diverse model, such as a Transformer or the Autoencoder+MLP architecture.

Week 4: Final Model Selection, Refinement, and Submission Strategy

The final week is for finalizing the models, polishing the ensemble, and making strategic submission choices.

- **Days 22-25:** Finalize the portfolio of 2-3 diverse models. Re-tune the ensemble weights with the inclusion of the third model. Analyze the correlation of predictions between the models; lower correlation is better for ensembling.
- **Days 26-28:** Conduct final, long training runs of the best individual models and the best ensemble on the full training data. Perform final sanity checks on the code and predictions.
- **Days 29-30:** Carefully select the two final submissions based on a combination of the highest CV scores, model diversity, and stability across validation folds. Complete the documentation and model write-up.

Table 3: 30-Day Action Plan Summary

Week	Days	Primary Focus	Key Deliverables
1	1-7	Foundation & Baseline	EDA Report, Working CPCV Framework, Baseline LGBM with CV Score
2	8-14	Feature Engineering & Model Diversity	Expanded Feature Set (>200 features), Prototype LSTM Model
3	15-21	Ensembling &	First Ensemble

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

		Optimization	(LGBM+LSTM), Optuna Tuning Jobs Running, Prototype 3rd Model
4	22-30	Finalization & Submission	Final Tuned Models, Optimized Ensemble, Selection of 2 Final Submissions

Winning Insights: The Grandmaster's Edge

The gap between a top 10% finish and a winning position is often closed by applying a deeper, more strategic layer of thinking to the problem. These are the non-obvious techniques and mental models that can provide a decisive edge.

Lessons from Legends: Distilled Wisdom from Jane Street, Two Sigma, and Optiver Competitions

Analyzing the winning solutions from past, high-stakes financial competitions provides a proven playbook of effective strategies.

- Jane Street Market Prediction [36]:
  The first-place solution highlighted the power of sophisticated, non-linear feature extraction. Their use of a supervised autoencoder to learn a compressed data representation before feeding it to an MLP demonstrates that automated feature learning can outperform purely manual feature engineering. Other key techniques included treating the problem as a multi-label classification task (predicting several different return horizons simultaneously) to stabilize the model, and using the magnitude of returns as sample weights to force the model to focus on the most impactful trading opportunities.
- Two Sigma Financial Modeling Challenge [25]:
  The key takeaway from this competition was the critical importance of modeling market

Downloaded by ?? ? (gaolijin0807@gmail.com)

regimes. Winners explicitly partitioned their data based on volatility and trained different models for different market conditions. This competition also served as a harsh lesson in not overfitting the public leaderboard; the market regime changed between the public and private test sets, causing a massive leaderboard shake-up. Finally, in time-series API competitions, code robustness is a competitive advantage in itself, as many competitors are eliminated due to runtime errors.

- Optiver Realized Volatility Prediction [45]:
  This competition showed that for certain types of financial data (in this case, high-frequency order book data), deep learning models like 1D CNNs and RNNs applied directly to raw data sequences can uncover patterns that are difficult to capture with handcrafted features. While the current competition uses daily data, the underlying principle—that deep learning architectures can be powerful feature extractors—remains valid.

The Secret Sauce: Unconventional Techniques to Climb from Silver to Gold

These are advanced, high-impact techniques that directly address the specific nuances of the competition's metric and the nature of financial data.

- **Volatility Targeting:** This is the single most important strategy derived from the analysis of the evaluation metric. A naive model outputs a prediction, which is then mapped to a 0 or 2 investment. A winning model will implement a control system. The final prediction should be a function of the forecasted return, scaled by a factor α, such that the resulting portfolio's volatility on the validation set is just under the 1.2x market volatility cap. This factor α can be solved for numerically on the out-of-fold validation data. This transforms the model from a simple predictor into a risk-managed system that is explicitly optimizing for the competition's constrained objective.[9]
- **Prediction Post-processing and Thresholding:** The mapping from the model's continuous return forecast to the final discrete prediction is a critical, and tunable, step. Instead of a simple if prediction > 0 rule, a threshold should be introduced: if prediction > threshold. This threshold becomes a hyperparameter that can be optimized on the validation set. By setting a small positive threshold, the strategy can filter out low-confidence positive predictions that contribute more to volatility (risk) than to mean return, thereby directly improving the Sharpe ratio.
- **Feature Neutralization:** A sophisticated technique used in institutional portfolio management that can be adapted for machine learning models. Certain powerful features, while predictive on average, may have unstable relationships with the target that can change dramatically during market regime shifts. To build a more robust model, one can neutralize the predictions with respect to such a feature. The process involves

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

taking the model's final out-of-fold predictions and fitting a simple linear regression: predictions ~ unstable_feature. The residual from this regression, predictions - fitted_values, represents a new set of "neutralized" predictions that are, by construction, orthogonal to (uncorrelated with) the unstable feature. Using these neutralized predictions can lead to a model that is more resilient to structural breaks in the market, providing a crucial edge in robustness.

Works cited

1. www.investopedia.com, accessed on September 23, 2025, https://www.investopedia.com/terms/s/sharperatio.asp#:~:text=The%20Sharpe%20ratio%20is%20one,expected%20variability%20of%20such%20returns.
2. Sharpe ratio - Wikipedia, accessed on September 23, 2025, https://en.wikipedia.org/wiki/Sharpe_ratio
3. Calculate the Sharpe Ratio to Gauge Risk | Charles Schwab, accessed on September 23, 2025, https://www.schwab.com/learn/story/calculate-sharpe-ratio-to-gauge-risk
4. The Sharpe Ratio - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/dimkapoulas/the-sharpe-ratio
5. Hull Tactical: Complete EDA Deep Dive - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/ahsuna123/hull-tactical-complete-eda-deep-dive
6. William F.Sharpe & The Sharpe Ratio - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/mpwolke/william-f-sharpe-the-sharpe-ratio
7. Jane Street Market Prediction | Kaggle, accessed on September 23, 2025, https://www.kaggle.com/competitions/jane-street-market-prediction
8. Hull Competition Sharpe - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/metric/hull-competition-sharpe
9. Hull Tactical - Max Leaderboard - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/mbrosseau/hull-tactical-max-leaderboard
10. Hull Tactical - Market Prediction -ProbingLB_02 - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/youneseloiarm/hull-tactical-market-prediction-probinglb-02
11. Time Series Forecasting: Mastering Predictive Sales Models [2025] - Forecastio, accessed on September 23, 2025, https://forecastio.ai/blog/time-series-forecasting
12. Time Series Analysis: Definition, Types, Techniques, and When It's Used - Tableau, accessed on September 23, 2025, https://www.tableau.com/analytics/what-is-time-series-analysis
13. Feature Engineering - Overview, Process, Steps - Corporate Finance Institute, accessed on September 23, 2025, https://corporatefinanceinstitute.com/resources/data-science/feature-engineering/
14. What are the most effective feature engineering techniques for stock market

Downloaded by ?? ? (gaolijin0807@gmail.com)

prediction using machine learning? - Consensus, accessed on September 23, 2025, https://consensus.app/search/what-are-the-most-effective-feature-engineering-te/yEqat2ZzSnish400SBaOWg/
15. Survey of feature selection and extraction techniques for stock market prediction - PMC, accessed on September 23, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC9834034/
16. Time Series using LightGBM - GeeksforGeeks, accessed on September 23, 2025, https://www.geeksforgeeks.org/machine-learning/time-series-using-lightgbm/
17. Feature engineering for time-series data - Statsig, accessed on September 23, 2025, https://www.statsig.com/perspectives/feature-engineering-timeseries
18. Feature Engineering in Trading: Turning Data into Insights - LuxAlgo, accessed on September 23, 2025, https://www.luxalgo.com/blog/feature-engineering-in-trading-turning-data-into-insights/
19. Useful features in Predicting Stock Prices - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/riteshsinha/useful-features-in-predicting-stock-prices
20. GARCH Model: Definition and Uses in Statistics - Investopedia, accessed on September 23, 2025, https://www.investopedia.com/terms/g/garch.asp
21. Mastering Volatility Forecasting with GARCH Models - Medium, accessed on September 23, 2025, https://medium.com/@sheikh.sahil12299/mastering-volatility-forecasting-with-garch-models-a-deep-dive-into-financial-market-dynamics-8df73c037b7e
22. Predicting Volatility: GARCH vs XGBoost - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/code/lucastrenzado/predicting-volatility-garch-vs-xgboost
23. Market Microstructure and Algorithmic Trading - Master of Science in Computational Finance, accessed on September 23, 2025, https://www.cmu.edu/mscf/academics/curriculum/46982-market-microstructure-and-algorithmic-trading.html
24. What are the most effective feature engineering techniques for stock market prediction using machine learning? - Consensus, accessed on September 23, 2025, https://consensus.app/search/what-are-the-most-effective-feature-engineering-te/DF5TnKcGQSy2_AskhA9_DQ/
25. two sigma | Stephen Smith's Blog, accessed on September 23, 2025, https://smist08.wordpress.com/tag/two-sigma/
26. Two Sigma Research: A Machine Learning Approach to Regime Modeling, accessed on September 23, 2025, https://www.venn.twosigma.com/resources/streetview-a-machine-learning-approach-to-regime-modeling
27. How to Use XGBoost and LGBM for Time Series Forecasting? - 365 Data Science,

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)

accessed on September 23, 2025, https://365datascience.com/tutorials/python-tutorials/xgboost-lgbm/
28. Cross Validation in Time Series. Cross Validation: | by Soumya Shrivastava | Medium, accessed on September 23, 2025, https://medium.com/@soumyachess1496/cross-validation-in-time-series-566ae4981ce4
29. A Novel Variant of LSTM Stock Prediction Method Incorporating ..., accessed on September 23, 2025, https://www.mdpi.com/2227-7390/12/7/945
30. Stock Price Prediction using LSTM and its Implementation - Analytics Vidhya, accessed on September 23, 2025, https://www.analyticsvidhya.com/blog/2021/12/stock-price-prediction-using-lstm/
31. Stock Market Predictions with LSTM in Python - DataCamp, accessed on September 23, 2025, https://www.datacamp.com/tutorial/lstm-python-stock-market
32. How to Develop LSTM Models for Time Series Forecasting ..., accessed on September 23, 2025, https://machinelearningmastery.com/how-to-develop-lstm-models-for-time-series-forecasting/
33. Time Series Forecasting using Transformer Neural Network - GitHub, accessed on September 23, 2025, https://github.com/rezaAdinepour/Time-Series-Forecasting
34. A Closer Look at Transformers for Time Series Forecasting: Understanding Why They Work and Where They Struggle | OpenReview, accessed on September 23, 2025, https://openreview.net/forum?id=kHEVCfES4Q¬eId=mrNbq9EkQa
35. Transformers in Time Series: A Survey - IJCAI, accessed on September 23, 2025, https://www.ijcai.org/proceedings/2023/0759.pdf
36. Yirun's Solution (1st place): Training Supervised Autoencoder with ..., accessed on September 23, 2025, https://www.kaggle.com/c/jane-street-market-prediction/discussion/224348
37. Hyperparameter tuning with Optuna in PyTorch - GeeksforGeeks, accessed on September 23, 2025, https://www.geeksforgeeks.org/deep-learning/hyperparameter-tuning-with-optuna-in-pytorch/
38. optuna/optuna: A hyperparameter optimization framework - GitHub, accessed on September 23, 2025, https://github.com/optuna/optuna
39. Optuna: A hyperparameter optimization framework — Optuna 4.5.0 documentation, accessed on September 23, 2025, https://optuna.readthedocs.io/
40. Federal Reserve Economic Data (FRED) - Appalachian State University Libraries, accessed on September 23, 2025, https://library.appstate.edu/research/databases/federal-reserve-economic-data-fred
41. Federal Reserve Economic Data - Wikipedia, accessed on September 23, 2025, https://en.wikipedia.org/wiki/Federal_Reserve_Economic_Data
42. Federal Reserve economic data (FRED) - Map and Data Library - University of Toronto, accessed on September 23, 2025,

Downloaded by ?? ? (gaolijin0807@gmail.com)

https://mdl.library.utoronto.ca/collections/numeric-data/federal-reserve-economic-data-fred
43. Fear at new low on D-Street. Is the worst of declines over?, accessed on September 23, 2025, https://m.economictimes.com/markets/stocks/news/indias-vix-slips-to-lifetime-lows-traders-see-limited-near-term-risks/articleshow/123932747.cms
44. VIX Index - Cboe Global Markets, accessed on September 23, 2025, https://www.cboe.com/tradable_products/vix/
45. No Feature Engineering NN Only Solution | Kaggle, accessed on September 23, 2025, https://www.kaggle.com/competitions/optiver-realized-volatility-prediction/writeups/stonks-no-feature-engineering-nn-only-solution
46. Two Sigma Financial Modeling Challenge - Kaggle, accessed on September 23, 2025, https://www.kaggle.com/c/two-sigma-financial-modeling/leaderboard
47. Cross-validation (statistics) - Wikipedia, accessed on September 23, 2025, https://en.wikipedia.org/wiki/Cross-validation_(statistics)
48. What is Combinatorial Purged Cross-Validation for time series data?, accessed on September 23, 2025, https://stats.stackexchange.com/questions/443159/what-is-combinatorial-purged-cross-validation-for-time-series-data
49. skfolio.model_selection.CombinatorialPurgedCV — skfolio, accessed on September 23, 2025, https://skfolio.org/generated/skfolio.model_selection.CombinatorialPurgedCV.html
50. [D] Benefits of Purged CV in Time Series? : r/MachineLearning - Reddit, accessed on September 23, 2025, https://www.reddit.com/r/MachineLearning/comments/1j392dd/d_benefits_of_purged_cv_in_time_series/

This document is available on studocu

Downloaded by ?? ? (gaolijin0807@gmail.com)